Algorytmy i struktury danych

Kolejki priorytetowe i kopce

Interaktywna wizualizacja krok po kroku

Łukasz Molis · nr indeksu 91612

Rok akademicki 2026/2027

1. Problem i cel

Co wyjaśnia

Kolejka priorytetowa zwraca zadania wg ważności; mniejsza liczba = wcześniej.

Trzy realizacje: tablica nieuporządkowana, uporządkowana, kopiec binarny.

Odbiorca

Studenci uczący się struktur danych.

Znają notację O oraz tablice i drzewa.

Czego się nauczą

Porównać koszt dodawania i pobierania.

Zrozumieć sift-up i sift-down.

2. Zakres projektu

Obowiązkowe podtematy

Tablica nieuporządkowana

Tablica uporządkowana

Kopiec binarny (min)

Do 31 zadań, priorytet 0–9, reguła (priorytet, kolejność dodania)

Wykonane funkcje

Dodaj / pokaż min / pobierz min

Krok, Start/Pauza, Reset, tempo

3 scenariusze + dane własne

Pseudokod z wyróżnioną linią, 3 pytania, porównanie, 10 testów, walidator

3. Algorytmy i mechanizmy

Wariant	Dodanie	Pobranie min	Kiedy stosować
Nieuporządkowana	O(1), 0 porównań	O(n), skan n−1	dodajemy częściej niż pobieramy
Uporządkowana	O(n) porównań + przesunięć	O(1), element[0]	mały zbiór, częste pobieranie
Kopiec min	O(log n), sift-up	O(log n), sift-down	obie operacje częste, duży zbiór

```
siftUp(i): while i>0 and heap[i] < heap[parent]: swap; i = parent
```

4. Struktury danych

Element: **{id, priority}** · porządek: para (priorytet, id) · tablica dynamiczna do 31 elementów

Nieuporządkowana

Brak porządku; kolejność dodania.

Uporządkowana

Niezmiennik: sortowanie wg (priority, id).

Kopiec

Rodzic ≤ dzieci; dzieci węzła i: 2i+1 i 2i+2.

Uzasadnienie: tablica daje lokalność pamięci, a kopiec bez wskaźników — rodzica i dzieci wylicza się arytmetycznie. Stan: liczniki porównań, przesunięć i swapów.

5. Technologie i architektura

Czysty HTML + CSS + JavaScript, jeden plik, zero bibliotek, praca lokalna.

Generatory function* (kroki algorytmów)

Kolejka playQueue

Silnik: processQueue → executeStep

Render DOM: tablica, drzewo, pseudokod, baner

Dodatkowo: runTests i klasa Validator, niezależne od algorytmów.

6. Realizacja i interfejs

1. Trzy algorytmy jako generatory kroków

2. Silnik: pauza, krok, tempo

3. Wizualizacja tablic i drzewa z legendą

4. Scenariusze, nauka, testy, walidator

Każdy krok powstaje z wykonania kodu — wizualizacja to nie nagrana animacja.

[Wstaw własny zrzut ekranu aplikacji]

7. Testy i porównanie

10/10

testów PASS, w tym 2 stany pośrednie

n = 31	Nieupor.	Upor.	Kopiec
Dodanie 1 el.	0	ok. 15	do 5
Pobranie 1 el.	30	1	do 5
Suma 31 pobrań	465	31	ok. 150–160

Liczniki: porównania (wszystkie warianty), przesunięcia (tablica uporządkowana), swapy (kopiec) — mierzą rzeczywiste operacje w kodzie.

8. Wnioski i ograniczenia

Co wykazano

Kompromis dodawanie–pobieranie.

Kopiec: O(log n) na operację, najlepszy ogólnie.

Ograniczenia

Maks. 31 zadań, priorytet 0–9.

Brak zmiany priorytetu.

Kopiec bez jawnych krawędzi.

Wkład: własna implementacja 3 algorytmów, silnik krokowy, testy, walidator. **Źródła:** Cormen i in. „Wprowadzenie do algorytmów”; Sedgewick, Wayne „Algorithms”; MDN. **AI:** pomoc w kodzie i dokumentacji; wyniki zweryfikowane ręcznie (testy w Node.js), algorytmy napisane jawnie.

9. Plan demonstracji

1

Moduły

Trzy tablice, porównanie i testy, nauka

2

Wykonanie krokowe

Dodaj, Krok, Start; pseudokod i baner

3

Zmiana danych

Scenariusz, własny priorytet

4

Przypadek brzegowy

Równe priorytety (FIFO) lub pusta struktura

Następnie: uruchomienie aplikacji.